

18-427/727: Board-Level RF Systems for IoT

LAB04: SDRs & Digital Modulation

Writeup Due: Monday, November 3rd, 2025

How to submit labs:

Create a word document or PDF that answers the questions presented below. Include any screenshots or figures as requested. **Please provide a descriptive caption for each figure!**

Question:	1	2	3	4	Total
Points:	0	10	50	40	100
Score:					

Lab Outline

This lab will serve as an introduction to software-defined radios (SDRs). Each of you will receive an RTL-SDR, which functions as a receiver. You will use GNU Radio to simulate BPSK signals, as well as detect and demodulate FM and QPSK signals in this lab.

1. Setup
2. Simulation in GNU Radio
3. FM Broadcast
4. QPSK Broadcast

Equipment Required

- Laptop - to run GNU Radio and to connect to the RTL-SDR.
- RTL-SDR - SDR receiver, provided by teaching team. One per lab group.
- GNU Radio - for this lab, you will need to install GNU Radio in order to interface with your RTL-SDR and to collect RF signal samples. More on that in the Setup section!

1. Setup

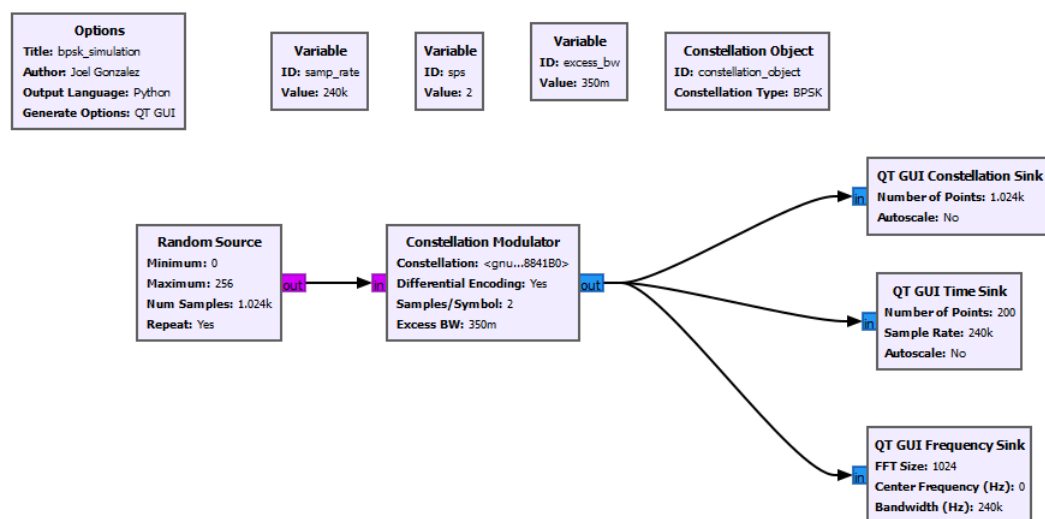
For this lab, we will use GNU Radio. The GNU Radio installation process can be found here: <https://wiki.gnuradio.org/index.php/InstallingGR>. We've found it easier to run GNU Radio on a Windows computer compared to a Mac, so if someone in your group has Windows, we recommend running the software on their computer. Please reach out to the teaching staff if you have any trouble getting it to run!

The teaching staff will give you an RTL-SDR and a whip antenna. This is a small, cheap device that functions as a receiver. It plugs into your laptop via USB, and it has a port to which you may connect an antenna. You can follow this link to complete the setup for the RTL-SDR: <https://www.nooelec.com/store/qs>.

2. Simulation in GNU Radio

GNU Radio is a free software that lets you easily program software-defined radios. GNU Radio provides the framework to create a signal flow graph, which lets you decide how RF signals in your radio will be processed. GNU Radio Companion then produces C++ or Python code, which can be run on your SDR. Various radios such as the RTL-SDR and USRP B210 (which we will use in this lab) are supported. This is quite useful for quickly testing out algorithms on your SDR hardware!

We will start off by running a simple simulation of a BPSK modulation scheme in GNU Radio to help familiarize yourself with the software. Download **BPSK-Simulation.grc**, which is the model shown below:



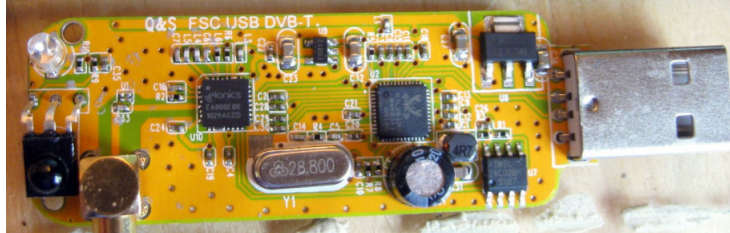
This flow graph uses the **Constellation Modulation** block to perform BPSK. This maps the data, which is a random sequence of bits, to one of two points on a constellation diagram. Note how we can define variables in the model, which are then used in the blocks in the flow graph itself. In the next sections, we'll see how to connect the flow graph to an actual SDR that is connected via USB to your computer. For now, feel free to click around and get comfortable with the UI of GNU Radio! You may also want to check the following link from the GNU Radio wiki to get a better idea of what's happening: https://wiki.gnuradio.org/index.php/Simulation_example:_BPSK_Demodulation

10 pts

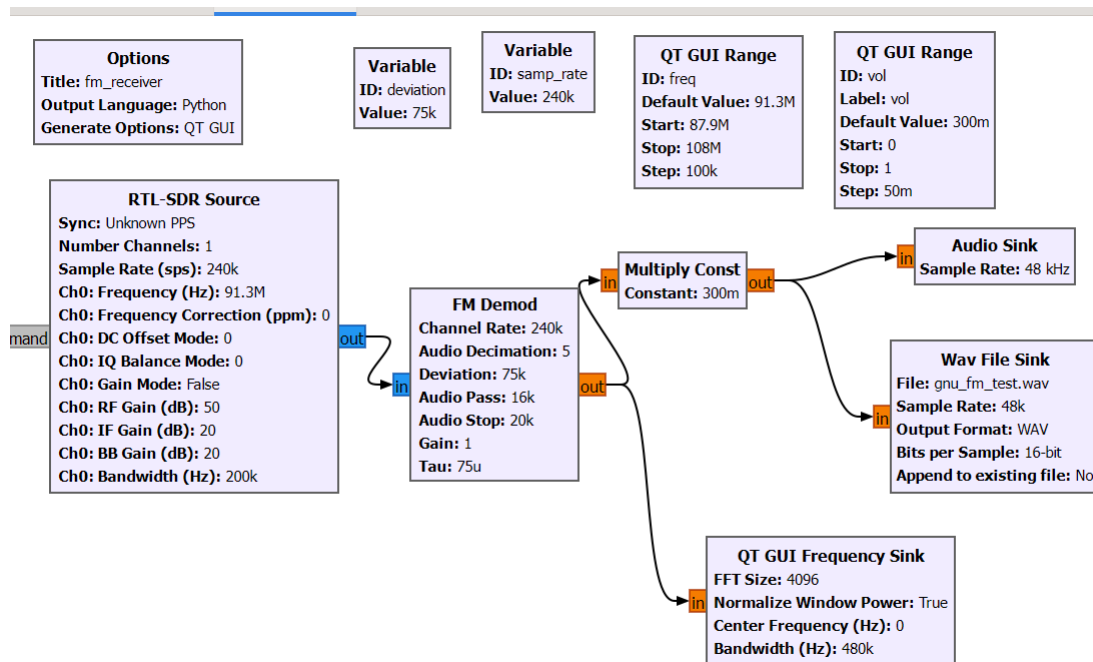
- 2.1** Run the model and take a screenshot of the GUI that pops up. Explain what the IQ plot on the GUI is telling us.

3. FM Broadcast

This part of the lab will introduce the RTL-SDR, using it to demodulate readily-available FM broadcast signals. The RTL-SDR is a cheap, receive-only device that connects to your computer over USB. It can capture up to 1.2 MHz of spectrum at once, with a frequency coverage of 24 MHz to 1.8 GHz. We are going to use it to receive and demodulate local FM radio broadcasts, using GNU Radio's support for the device.



We will use GNU Radio to collect the IQ samples from the RTL-SDR and process them. Below is the GNU Radio model that we will use, **FM-Receiver.grc**:



Here's a breakdown of the individual blocks:

- The **RTL-SDR Source** represents the SDR in software. The **Data** output of the RTL-SDR Receiver block consists of complex-valued IQ samples collated into a user-selected frame length. This model is configured to a sample rate of 240 kHz, and produces 10000-sample frames for analysis.
- The **FM Demod**'s task is self-explanatory, and results in the **Audio Signal**. It has a low-pass filter included, which processes the demodulated signal to the audio frequency range.
- The low-pass filtered audio is passed to a **Frequency Sink** to show the spectrum of the output signal.
- The **Multiply Constant** multiplies the signal value by a factor; this block functions as volume control for the output audio.
- The final audio signal is output to your devices' speakers via the **Audio Sink**, while the **Wav File Sink** block records and saves the audio to a designated .wav file.

The simulation is tuned to 91.3 MHz WYEP, the self-proclaimed “Best Radio Station in Pittsburgh,” a commercial station with strong signal on campus. Connect your RTL-SDR to your laptop and open the **FM-receiver.grc** file using GNC Radio Companion. Run the program, and listen to the audio output while observing the frequency plot. The results will be recorded and saved as **gnu_fm_test.wav**.

10 pts

3.1 The commercial FM broadcast band is divided into channels that are 200 kHz wide as you can observe in the spectrum of the **FM Signal**. There’s actually a lot more going on in each broadcast band than simply transmitting one audio signal. Please include a screenshot of the spectrum of the demodulated Baseband Signal and annotate the signal’s components. You may find the following reference useful: https://www.sigidwiki.com/wiki/FM_Broadcast_Radio.

15 pts

3.2 Listen to the **gnu_fm_test.wav** audio file and observe the **QT GUI Frequency Sink** block. Why were the low-pass filter edges set the way they were?

15 pts

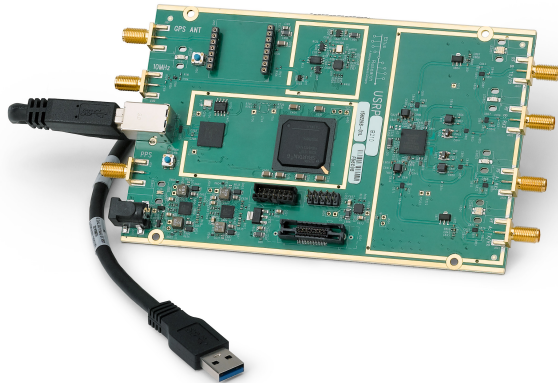
3.3 Describe how you might be able to isolate the Left and Right (L and R) channels from the $L + R$ and the $L - R$ signals (i.e. the “sum” and “difference” signals) to generate a stereo audio signal. Please include in your answer why there is a pilot tone at 19 kHz.

10 pts

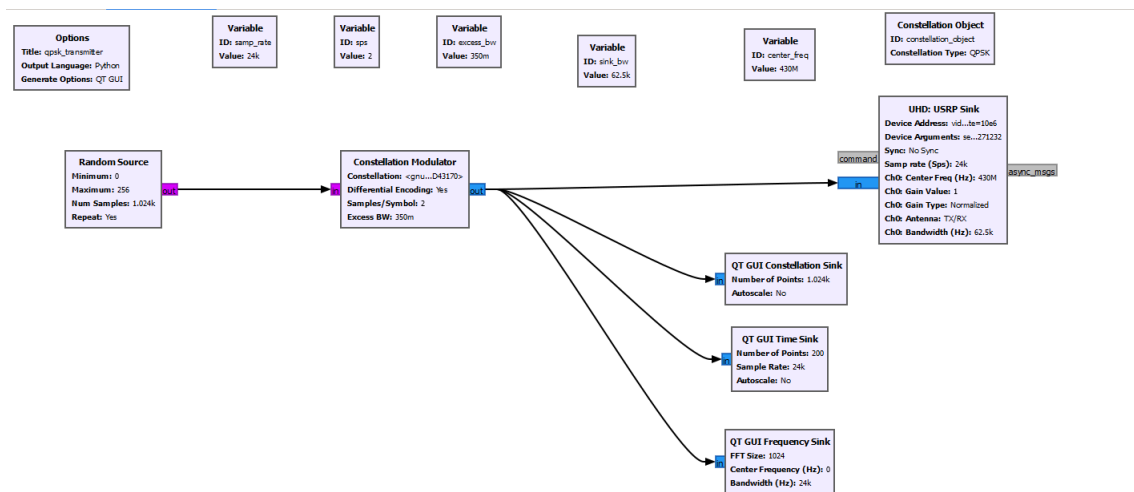
3.4 Now change your simulation to tune in to WRCT, the CMU campus radio station at 88.3 MHz, and please paste a screenshot of the **Baseband Signal**. Describe all the ways you can observe that the baseband signal differs from the commercial WYEP station.

4. QPSK Broadcast

For this part of the lab, we will be broadcasting a QPSK signal that you will receive and demodulate using your RTL-SDR. We will provide you with a GNU model for demodulation, and you will tweak parameters to better understand how the QPSK demodulator is working. For broadcasting, we will be using a USRP B210 transceiver:

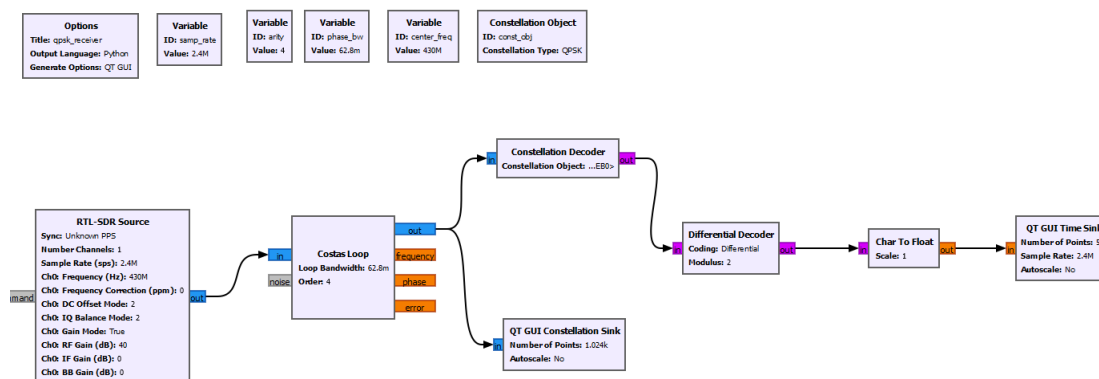


This device can operate up to 6 GHz and can easily transmit 10 dBm of power. Here, we'll be using it to transmit QPSK signals inside the classroom. Here is the GNU Radio model that we are using for transmission, called **QPSK-Transmitter.grc**:

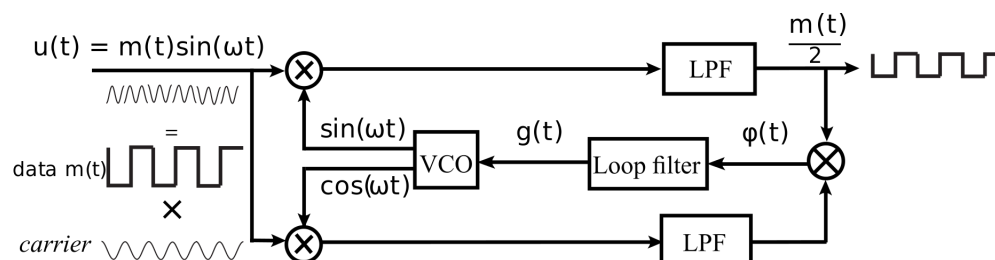


In this model, we are again using the **Constellation Modulation** block to perform QPSK. This maps the data to one of four points on a constellation diagram. After QPSK modulation, this data is sent to an SDR **Sink** block, which in our case is the USRP B210.

On the receive side, **QPSK-Receiver.grc** is the model that you will be using with your RTL-SDR:



This model uses a **Costas loop** to recover the carrier frequency:



This is based on a phase-locked loop (PLL), which is a type of feedback control loop that relates the output signal to the phase of the input signal. A Costas loop uses a voltage-controlled oscillator (VCO) to try and generate a sinusoid that matches the frequency of the input signal. You may find this article on the Costas loop to be of interest: https://en.wikipedia.org/wiki/Costas_loop

Using this control loop, the RTL-SDR will attempt to lock onto the carrier frequency. Once this frequency is known, it uses the **Constellation Decoder** block to demodulate the PSK signal. From here, we're able to see how the received data looks on a constellation diagram.

10 pts 4.1 Run the model and take a screenshot of your QPSK constellation diagram.

10 pts 4.2 Note that we set our sample rate to 2.4 MHz. What happens if we decrease this sampling rate by a factor of 10 (240 kHz)? Provide an explanation for what changes you see on the constellation diagram.

20 pts 4.3 Note the *phase_bw* variable in the receiver model. This controls the bandwidth of the low-pass filter in the Costas loop. Try changing the loop bandwidth. Provide an explanation for what happens to the constellation diagram when you significantly increase or decrease the loop bandwidth.

Reflections

Please complete our [post-lab survey](#) for 5 points of extra credit! Attach a photo of the form completion page to your lab submission.